

Шпаргалка по проектированию систем

Порядок разбора задачи, оценка нагрузки, масштабирование и кеширование, базы данных и очереди, устойчивость к отказам, наблюдаемость, компромиссы и типовые схемы. Годится и для собеседования, и для реальной работы.

Порядок разбора

1. Требования
Что система делает, кто ей пользуется, какие сценарии главные
2. Нефункциональные требования
Нагрузка, допустимые задержки, доступность, сохранность данных, стоимость
3. Оценка масштаба
Запросы в секунду, объём данных, рост за год — числа определяют архитектуру
4. Внешний интерфейс
Основные операции API и модель данных
5. Крупная схема
Клиенты, балансировщик, службы, хранилища, очереди
6. Узкие места
Где закончится запас и что делать: кеш, шардирование, асинхронность
7. Компромиссы
Назвать альтернативы и объяснить выбор — это и есть суть проектирования

Оценка «на салфетке»

Сутки
 $86\,400 \text{ секунд} \approx 10^5$; миллион событий в сутки ≈ 12 в секунду

Пиковая нагрузка
Обычно в 2–5 раз выше средней

Объём данных
Число записей $\times$ размер записи $\times$ срок хранения $\times$ коэффициент репликации

Порядки задержек
Память — наносекунды, SSD — десятки микросекунд, запрос в дата-центре — доли миллисекунды, между континентами — сотни миллисекунд

Правило 80/20
Небольшая доля данных даёт основную часть обращений — отсюда эффективность кеша

Масштабирование

- Вертикальное
Мощнее машина: просто, но упирается в потолок и в единственную точку отказа
- Горизонтальное
Больше машин: требует отсутствия состояния в службах и балансировки
- Балансировка
По кругу, по числу соединений, по хешу — плюс проверки работоспособности
- Службы без состояния
Сессии в общем хранилище, а не в памяти процесса
- Разделение чтения и записи
Реплики для чтения снимают нагрузку с основной базы
- CDN
Статика ближе к пользователю; разгружает приложение

Кеширование

- Уровни
Браузер, CDN, обратный прокси, кеш приложения, кеш базы
- Cache-aside
Приложение само читает кеш и наполняет его при промахе — самая частая схема
- Write-through / write-behind
Запись через кеш сразу / с задержкой
- Вытеснение
LRU, LFU, по сроку жизни
- Инвалидация
Самая сложная часть: срок жизни, событие изменения или версия в ключе
- Лавина промахов
Одновременное истечение множества ключей роняет базу: разброс сроков и блокировка на перезаполнение

Базы данных

Реляционные

Связи, транзакции, произвольные запросы; выбор по умолчанию

Документные

Гибкая схема, данные читаются целым документом

Ключ — значение

Простой доступ, очень высокая скорость

Колоночные

Аналитика по большим объёмам

Временных рядов

Метрики и события с отметками времени

Репликация

Копии для отказоустойчивости и чтения; у асинхронной есть отставание

Шардирование

Разделение данных по ключу; усложняет запросы между шардами

Индексы

Ускоряют чтение, замедляют запись, занимают место

Согласованность

- **Теорема CAP:** при сетевом разделении приходится выбирать между согласованностью и доступностью.
- **Строгая согласованность** — чтение всегда видит последнюю запись; дороже и медленнее.
- **Согласованность в конечном счёте** — реплики сходятся со временем; годится для лент, счётчиков, поиска.
- **Чтение своих записей** — компромисс: пользователь видит собственные изменения сразу.
- **Идемпотентность** — повтор операции не меняет результат; обязательна при повторных попытках.

Асинхронность

Очередь задач

Долгие операции выносятся из запроса: письма, отчёты, обработка файлов

Шина событий

Службы публикуют факты, подписчики реагируют — слабая связанность

Гарантии доставки

«Хотя бы раз» — норма; дубликаты лечит идемпотентность

Очередь недоставленных

Сообщения, которые не удалось обработать, откладываются для разбора

Обратное давление

Ограничение приёма, когда обработчики не справляются

Устойчивость к отказам

Тайм-ауты

Любой сетевой вызов должен иметь предел ожидания

Повторы с выдержкой

Растущие интервалы и случайный разброс, иначе клиенты ударят разом

Предохранитель

После очереди ошибок перестать звать неисправную службу и попробовать позже

Переборки

Разделение ресурсов, чтобы отказ одной части не утянул остальные

Плавная деградация

Выдать частичный или устаревший ответ вместо ошибки

Ограничение частоты

Защита от перегрузки и злоупотреблений

Наблюдаемость

- **Метрики** — числовые ряды: частота запросов, доля ошибок, задержки. Смотрите перцентили (p95, p99), а не среднее.
- **Журналы** — структурированные записи с идентификатором запроса.
- **Трассировка** — путь запроса через службы; показывает, где потеряно время.
- **SLI и SLO** — что измеряем и какой уровень обещаем; бюджет ошибок связывает надёжность и темп изменений.
- **Оповещения** — по симптомам, важным пользователю, а не по каждому скачку метрики.

Компромиссы

- **Монолит или службы:** монолит проще и быстрее в разработке; службы дают независимые выкладки ценой сложной эксплуатации.
- **Синхронно или асинхронно:** простой ответ против устойчивости и пропускной способности.
- **Нормализация или денормализация:** целостность против скорости чтения.
- **Согласованность или доступность:** зависит от цены устаревших данных в вашей задаче.
- **Купить или написать:** готовое решение обычно дешевле, чем кажется.

Типовые схемы

- **Сокращатель ссылок:** генерация короткого ключа, хранилище «ключ — значение», кеш, перенаправление 301/302.
- **Лента новостей:** сборка при чтении или рассылка при записи; для популярных авторов — смешанный подход.
- **Чат:** WebSocket, хранение сообщений с ключом по беседе, счётчики непрочитанного.
- **Загрузка файлов:** прямая отправка в объектное хранилище по временной ссылке, обработка через очередь.
- **Поиск:** отдельный индекс, обновляемый событиями из основной базы.
- **Ограничитель частоты:** счётчики в общем кеше, алгоритмы «токенное ведро» или «скользящее окно».

Что помнить

- Начинайте с простого решения и усложняйте по мере появления реальных ограничений.
- Сначала измерение, потом оптимизация: узкое место редко там, где кажется.
- Каждая новая служба добавляет сетевые вызовы, сбои и работу по сопровождению.
- Явно называйте, чем жертвуете: решение без компромиссов обычно означает непонятый вопрос.
- Проектируйте под отказ: машины, сети и зависимости выходят из строя всегда.

Оригинальная шпаргалка на русском языке для репозитория [2web/cheat-sheet-pdf](#).